

Welcome to the team! Working with FPGAs—especially Xilinx SoCs and FPGAs—is incredibly rewarding, but it requires a shift from how you think about software. When we implement a protocol in hardware, we aren't just writing code to read bytes; we are building physical architectures that process data deterministically at line rate.

Today, we are diving into **ARINC 818-2** (Avionics Digital Video Bus). It's a high-speed, low-latency, fibre-channel-based protocol used for routing video in military and commercial aircraft cockpits (like radar displays, heads-up displays, and mission computers).

If you master the *methodology* of how to dissect ARINC 818, you will be able to pick up **any** high-speed serial protocol in the future (like PCIe, Serial RapidIO, or Ethernet) and build a production-ready FPGA project around it.

Here is our 4-phase roadmap to take you from a fresh graduate to an engineer who can independently own a protocol implementation.

Phase 1: The "Why" and the Document Anatomy (Day 1–3)

Goal: Understand the ecosystem and know exactly where to find answers without reading hundreds of pages front-to-back.

Before looking at a single line of HDL, you must understand the problem the protocol solves.

1. **The Big Picture:** ARINC 818 handles uncompressed, high-bandwidth digital video. Why uncompressed? Because in a fighter jet or a commercial airliner, visual latency must be near zero, and there can be no compression artifacts that misrepresent critical flight data.
2. **The Layered Approach:** Like most protocols, ARINC 818 maps to the OSI model.
 - **Physical Layer:** Fiber optics or copper, running at deterministic line rates (e.g., 1.0625, 2.125, 4.25, or 8.5 Gbps).
 - **Data Link Layer:** It borrows heavily from **Fibre Channel (FC-1 and FC-2)**. It uses 8b/10b encoding, ordered sets (special control characters like K28.5), and frames.
 - **Application Layer:** How the actual video pixels (RGB, YCbCr) are mapped into those frames.
3. **The Spec Is Your Bible:** Download the ARINC 818-2 specification. Don't read it like a novel. Flip straight to the **Frame Structure** and the **Object Definitions**. Keep a notebook of protocol-specific jargon (e.g., *SOFi* for Start of Frame Initial, *EOF* for End of Frame).

Phase 2: Dissecting the Protocol Architecture (Week 1)

Goal: Convert abstract text from the specification into concrete hardware block diagrams.

When you look at a protocol spec, your brain should automatically start translating paragraphs into hardware blocks. For ARINC 818, we break it down into three core elements:

1. The Container (The Frame)

A protocol is just a structured way of wrapping your data. ARINC 818 sends data in packets called "Frames."

- It starts with a 4-byte Start of Frame (SOF).
- It has a 24-byte Header (containing routing info, frame count, line number).

- It carries the Payload (the actual pixel data).
- It ends with a 4-byte Cyclical Redundancy Check (CRC) and a 4-byte End of Frame (EOF).

2. The Clocking and Alignment

High-speed data comes in as a single differential serial stream. The FPGA transceiver reconstructs a parallel bus (e.g., 20-bit or 40-bit wide) from this stream.

- How does the hardware know where a word starts? It looks for a specific "Comma" character (like the K28.5 symbol in 8b/10b encoding).
- You need to understand how the **Word Aligner** in the transceiver locks onto this character.

3. State Machines

Every protocol relies on a control backbone. For ARINC 818, you will map out a **Link Synchronization State Machine** (looking for valid sync sequences to declare the link "UP") and a **Video Reception State Machine** (waiting for SOF, counting lines, verifying CRC, detecting EOF).

Phase 3: The Xilinx Ecosystem & Hardware Mapping (Week 2)

Goal: Stop treating the FPGA as generic logic and leverage its dedicated silicon.

You rarely design a high-speed protocol entirely from scratch. We build on top of Xilinx primitives. For a Xilinx project, your implementation stack will look like this:

1. The Physical Media Attachment (PMA) & Physical Coding Sublayer (PCS)

We do not write RTL to sample a 4 Gbps serial line directly; regular FPGA fabric pins can't handle that speed. We use Xilinx Gigabyte Transceivers (**GTH/GTY** or **GTP**, depending on the FPGA family).

- You will use the **Xilinx Transceiver Wizard** (or Vivado IP Catalog) to instantiate the transceiver.
- Configure it to your specific ARINC 818 line rate, enable 8b/10b decoding, and set up the comma alignment pattern.

2. The Fabric Logic (Your Core RTL)

The output of the transceiver is now a stable, parallel clock and data bus (e.g., 32 bits of data at 106.25 MHz). This is where *your* RTL comes in. You will write:

- **The Parser/De-serializer:** Extracts the video header fields and validates them.
- **CRC-32 Engine:** Calculates the checksum on the fly to ensure no bit-flips occurred during flight.
- **Elastic FIFO:** The serial line clock is recovered from the incoming data and is asynchronous to your internal system/display clock. You must pass the pixel data through an asynchronous FIFO to safely cross clock domains (CDC).

Phase 4: Verification and Designing for Success (Week 3 onwards)

Goal: Ensure the design works in the real world under worst-case conditions, not just in a perfect simulation.

An engineer's quality is defined by how they test. To build an industry-grade ARINC 818 IP core, follow these verification steps:

- **Write a Directed Testbench:** Create a bus functional model (BFM) that feeds your RTL a perfectly valid ARINC 818 stream. Ensure your state machine locks on correctly.
- **Inject Errors:** This is what separates junior engineers from seniors. Modify your testbench to intentionally inject a wrong CRC, a dropped frame, or a corrupted K-character. Does your RTL hang? Or does it gracefully recover, flag an error bit, and wait for the next frame? High-reliability systems *must* handle errors gracefully.
- **Hardware In-the-Loop (HIL) Debugging:** Once we push to the Xilinx board, Vivado **ILA (Integrated Logic Analyzer)** is your best friend. Set up triggers on your link status signals and your state machine variables to capture real-time bugs on the hardware.

Your Golden Rule for the Future

Whenever you are handed a brand-new protocol standard in your career, use this exact 4-step mental framework:

1. **Identify the Core Unit:** What does a single packet/frame look like?
2. **Locate the Boundaries:** How does the hardware find the beginning and end of that unit? (Commas, preambles, start bits).
3. **Trace the Clock:** Where does the sampling clock come from, and where does the data need to go? (Clock Domain Crossing analysis).
4. **Map to Hardware:** Which parts belong in the Xilinx hard IP (transceivers/MGTs) and which parts must I write in Verilog/VHDL?

For the next two days, your only assignment is to read the ARINC 818-2 overview and sketch the layout of a single video frame on a piece of paper. Once you can explain that frame layout to me, we will open Vivado and configure our first Xilinx Transceiver wizard together. Let's get to work!

To truly master a protocol like **ARINC 818-2** and develop production-ready IP on a Xilinx FPGA, we need to move past high-level summaries and break down the exact technical mechanics. As your senior engineer, I am going to walk you through the architectural details, the hardware design choices, the specific Xilinx primitives, and the precise state machines you need to implement. Grab your notepad; here is the deep-dive technical manual for building an ARINC 818-2 system.

Section 1: Deep Dive into the ARINC 818 Specification

ARINC 818-2 is built entirely on top of the **Fibre Channel (FC)** physical and signaling standards. Specifically, it adopts Fibre Channel's FC-0 (Physical), FC-1 (Transmission Protocol), and parts of FC-2 (Signaling Protocol).

Unlike commercial video protocols (like HDMI or DisplayPort) which transmit continuous streams of clock and data over separate lanes, ARINC 818 packs video pixels into discrete packet structures called **Frames** and sends them over a single high-speed differential serial pair.

1. The Operational "Objects"

ARINC 818 does not just send raw pixels; it categorizes data into **Objects**. When you read the specification for a specific aircraft installation (defined in an ICD or *Interface Control Document*), you will see data structured into four primary object types:

- **Object 0 (Header/Metadata):** Transmitted at the start of a video frame. It contains setup parameters, camera angles, time stamps, or system status.
- **Object 1 (Video Data):** The main payload containing the actual lines of pixel data.
- **Object 2 (Audio Data):** Optional digital audio synchronized with the video stream.
- **Object 3 (Ancillary Data):** Auxiliary overlays, flight details, or secondary telemetry.

2. The Anatomy of an ARINC 818 Frame

Every transmission block is encapsulated by Fibre Channel standard "Ordered Sets"—which are 4-byte (32-bit) words consisting of a special control character (K-character) followed by 3 data bytes (D-characters).

A standard ARINC 818 video line frame looks exactly like this in memory:

1. **Start of Frame (SOF) – 4 bytes:** This is an ordered set. For the very first frame of a video packet, it uses **SOFi3** (Start of Frame Initial). For subsequent frames within the same video image, it uses **SOFn3** (Start of Frame Normal).
2. **Frame Header – 24 bytes:** Contains routing information. You must parse this to know what data is arriving:
 - Bytes 0-2: Destination ID.
 - Bytes 3-5: Source ID.
 - Byte 8: Type/Object ID (Tells your RTL if this is Object 0 metadata or Object 1 video line data).
 - Bytes 10-11: Sequence Number (Tracks frame integrity).
 - Bytes 14-15: Parameter Field (In Object 1, this holds the **Line Number** of the video image).
3. **Payload – Variable Bytes:** The actual uncompressed pixels (e.g., RGB 888, YCbCr 4:2:2).
4. **CRC – 4 bytes:** A 32-bit Cyclical Redundancy Check calculated over the Frame Header and Payload using the standard Fibre Channel polynomial:
5.
$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$
6. **End of Frame (EOF) – 4 bytes:** Another ordered set indicating termination. Uses **EOFn** (End of Frame Normal) for standard frames, or **EOFt** (End of Frame Terminate) for the final line of a video frame.

Section 2: Configuring the Xilinx Transceiver (MGT) Hard IP

We do not write Verilog to toggle lines at gigabit speeds. Xilinx FPGAs contain dedicated silicon blocks called **Multi-Gigabit Transceivers (MGTs)**—known as GTP, GTH, or GTY depending on whether you are using an Artix, Kintex, or Virtex/UltraScale+ chip.

Your first practical assignment is to configure the **Xilinx Transceiver Wizard** in Vivado. Here are the precise settings you must use:

1. Line Rate and Reference Clocks

ARINC 818 dictates deterministic link rates. Let's assume our ICD requires **4.25 Gbps**.

- You must route a highly stable, low-jitter external differential reference clock (e.g., 106.25 MHz or 212.5 MHz) to the dedicated **MGTREFCLK** pins.
- The internal Phase-Locked Loop (QPLL or CPLL) multiplies this reference clock to achieve the 4.25 GHz serialization rate.

2. The PCS (Physical Coding Sublayer) Configuration

Inside the transceiver block, you must enable two critical hardware sub-modules:

- **8b/10b Encoder/Decoder:** High-speed copper or fiber lines cannot tolerate long strings of continuous 1s or 0s because the receiver will lose DC balance and clock tracking. 8b/10b mapping converts every 8-bit byte into a 10-bit symbol, guaranteeing plenty of clock-rich transitions and maintaining a neutral Running Disparity (equal number of 1s and 0s).
- **Comma Detection and Word Alignment:** The serial data arrives bit-by-bit. The transceiver needs to know where the 10-bit boundary starts. You will configure the transceiver to look for the **K28.5** comma character (represented as **0101111100** or **1010000011**). When the transceiver senses this specific pattern, it automatically aligns its internal shift registers so that the parallel output bus aligns perfectly to the word boundary.

Section 3: Designing the Fabric RTL Architecture (Your Core Code)

Once the Xilinx transceiver handles serialization, 8b/10b decoding, and word alignment, it presents a parallel interface to the FPGA fabric. Typically, for a 4.25 Gbps line, it outputs a **32-bit wide data bus** accompanied by a recovered clock (**rxoutclk**) running at **106.25 MHz** ($4.25 \text{ Gbps} \div 40 \text{ bits of encoded data} = 106.25 \text{ MHz}$).

Now, your custom RTL takes over. Your design must be split into three modular pipelines.

1. Link Synchronization State Machine

Before processing video, your hardware must guarantee the physical link is healthy. You will write a finite state machine (FSM) that monitors the transceiver's output:

- **LOSS_OF_SYNC State:** The system powers up here. It waits to see continuous **IDLE** ordered sets (K28.5 followed by specific data bytes) coming from the transceiver.
- **LINK_ACQUIRING State:** If the RTL detects 4 consecutive valid IDLE words without 8b/10b errors, it moves forward.
- **LINK_READY State:** The link is declared active. If at any point the transceiver asserts an **rxnotintable** or **rxdisperr** (disparity error) error flag for multiple clock cycles, the FSM instantly drops back to **LOSS_OF_SYNC**, shielding downstream video processing from

junk data.

2. The Frame Parser FSM

Once the link is ready, your frame parser FSM continuously samples the 32-bit bus on the rising edge of `rxoutclk`.

[IDLE/IDLE] -> Detects SOFi/SOFn -> [PARSE_HEADER] -> [EXTRACT_PAYLOAD] -> [VALIDATE_CRC] -> Detects EOF -> Back to [IDLE]

Header Extraction: In the `PARSE_HEADER` state, count the incoming 32-bit words. Store the 24-byte header into registers. Look specifically at the **Object ID** to verify it's a video line, and extract the **Line Number** from the Parameter Field.

- **Payload Streaming:** Move to the `EXTRACT_PAYLOAD` state. Send the raw 32-bit pixel data directly into a pipeline while simultaneously feeding it into your CRC module.
- **CRC Verification:** ARINC 818 processes data fast, meaning your CRC calculation must be done *on the fly, in parallel*. You will implement a combinational 32-bit parallel CRC-32 calculator. When the FSM strikes the 4 bytes immediately preceding the `EOF`, it checks if your calculated CRC matches the transmitted CRC. If they match, a `Line_Valid` strobe is fired. If they mismatch, an error counter increments, and the bad line is flagged.

3. Clock Domain Crossing (CDC) & Display Driving

The recovered clock `rxoutclk` belongs to the incoming fiber link. However, your local display monitor or video processing pipeline runs on an independent local pixel clock (e.g., **148.5 MHz** for a standard 1080p display). **Never mix these clock domains directly.**

- You will instantiate a **Xilinx Independent Clock Asynchronous FIFO**.
- **Write Side:** Driven by `rxoutclk`. You write pixel data into the FIFO only when the Frame Parser is in the `EXTRACT_PAYLOAD` state.
- **Read Side:** Driven by your local display/system clock. Downstream logic reads from this FIFO, converts the raw pixels into a standard streaming protocol like **AXI4-Stream**, and feeds it to an onboard display controller IP or video processing block.

Section 4: Verification, Robustness, and Mission-Critical Engineering

In avionics engineering, "it works in simulation" is only 10% of the job. Military and civil aviation systems require extreme fault tolerance. To move from a junior to an advanced engineer, you must design your verification environment to thoroughly test failure recovery.

1. Behavioral Testbench & Error Injection

When you write your testbench in SystemVerilog or VHDL, create an autonomous **Bus Functional Model (BFM)** that acts as an ARINC 818 transmitter sending synthetic video patterns. Once your receiver parses it perfectly, start intentionally breaking things in your testbench to observe how your RTL handles exceptions:

- **Disparity/Bit Flip Injection:** Force a single bit flip inside a video frame. Verify that your CRC engine catches it, drops the specific line, and ensures the FSM does *not* freeze up but instead hunts for the next valid `SOF`.
- **Dropped Packets:** Delete an entire line from the testbench stream. Your receiver should recognize that the Line Number jumped from Line 42 straight to Line 44. It must flag a

missing line error to the processor and continue processing gracefully.

2. Hardware-In-the-Loop (HIL) Debugging

When we load the design onto our Xilinx evaluation board, you will instantiate a **Vivado ILA (Integrated Logic Analyzer)** core inside the fabric logic.

Tie the ILA probes to:

1. Your Link-Ready state variable.
2. The CRC error strobe.
3. The FIFO full/empty flags.

If the fiber-optic cable is disconnected or undergoes physical stress, you will capture the exact window of physical failure using the ILA triggers, allowing you to see how the hardware stabilizes itself when the connection is physically re-established.

Summary Action Plan for Tomorrow

When you come into the lab tomorrow morning, we are going to open Vivado. Here is your roadmap based on what we covered:

1. **Step 1:** We will open the Xilinx Transceiver Wizard to implement the physical layer (8b/10b, Comma alignment).
2. **Step 2:** You will draft the Verilog code for the Link Synchronization State Machine.
3. **Step 3:** We will write the Frame Parser together to unpack the Header and validate the CRC-32.

By following this precise sequence—breaking the protocol down into its physical, protocol, and application layers—you will develop an intuition for hardware design that makes mastering any future protocol an straightforward task. Let's get started!